OPM Flow — Native Windows Build

Building all targets from the `opm_flow_windows` harness (MSVC, hybrid MPI + OpenMP)

Prepared 2026-07-04, updated 2026-07-05 • Machine: Windows 11 Pro, 24 logical CPUs, 55.8 GB RAM • Toolchain: VS 2022 Build Tools 17.14 (MSVC 14.4x), CMake 4.x, Ninja, MS-MPI 10.1 • Build root: `C:\opm\opm_flow_windows`

1. Overview

The `opm_flow_windows` repository is a **build harness** that compiles OPM Flow as a **pure native Windows binary** (MSVC PE32+ — no WSL, Cygwin, or MSYS), in both serial and parallel (MPI + Zoltan) configurations. It does not contain the OPM / DUNE / Trilinos sources; its driver script clones them and builds everything in dependency order with the correct MSVC conformance flags and a small set of POSIX/Fortran compatibility shims.

This report first summarises the **documented build procedure** (from the repo's `README.md` and `BUILD_WINDOWS.md`), then records **what was executed** and the **findings**: **all 528 targets across the four OPM repositories build**, every unit test that executes passes (§4.4), and real simulations — including the Norne field case in hybrid MPI + OpenMP mode — run fine.

2. The documented build procedure

2.1 Prerequisites & toolchain (one-time)

- Windows 10/11 x64; `git` and `winget` on PATH; ~15 GB free disk; internet.
- VS 2022 C++ Build Tools + Windows 11 SDK + CMake + Ninja, via a single winget command.
- For MPI builds only: Microsoft MPI runtime (`Microsoft.msmpi`) + SDK (`Microsoft.msmpisdk`).
- **Location:** place the harness at a short **ASCII path without spaces, outside OneDrive-synced folders** (e.g. `C:\opm\opm_flow_windows`) — vcpkg cannot handle non-ASCII path components, and sync clients can lock files mid-build.

2.2 Sources & layout

The driver `build-all.ps1` lays out a single root and clones into it:

- **vcpkg** — the dependency manager.
- **DUNE 2.10.0** — `dune-common, dune-geometry, dune-istl, dune-grid` (pinned tag).
- **OPM** — `opm-common, opm-grid, opm-simulators` (+ `opm-upscaling` with `-Upscaling`).
- **Trilinos** — only for MPI builds, to build the Zoltan package.

> Build order (each step feeds the next):
> **DUNE:** dune-common → dune-geometry → dune-istl → dune-grid
> **OPM:** opm-common → opm-grid → opm-simulators → opm-upscaling

2.3 Third-party dependencies (vcpkg)

Built with the `x64-windows` triplet: `suitesparse-umfpack`, `fmt`, `lapack` (pulls OpenBLAS — provides BLAS + LAPACK, no Fortran compiler needed), and the Boost components OPM uses (`test, date-time, property-tree, mpl, range, spirit, filesystem, system`). The first run compiles OpenBLAS + SuiteSparse (20–40 min); vcpkg then caches them.

2.4 POSIX / Fortran compatibility shims

MSVC lacks a few POSIX headers and has no Fortran compiler; rather than patch every include, the harness puts small shim headers on the include path (`compat\include`):

- `unistd.h`, `getopt.h`, `sys/ioctl.h`, `sys/utsname.h` — POSIX stand-ins.
- `FCMacros.h` — Fortran name-mangling (gfortran/OpenBLAS ABI), normally generated by CMake's FortranCInterface.

2.5 Compile flags (applied to every module)

```
CXX: /permissive- /Zc:__cplusplus /Zc:preprocessor /bigobj /EHsc /wd4068
     -D_USE_MATH_DEFINES /DWIN32 /D_WINDOWS  /I <root>/compat/include
Config: -DCMAKE_BUILD_TYPE=Release  -G Ninja  (static OPM libs, C++20)
        vcpkg toolchain file + x64-windows triplet
```

OPM needs C++20, DUNE needs C++17. MPI/OpenMP/Fortran are off by default and enabled by switches.

2.6 MPI (Zoltan) & OpenMP

- **MPI** (`-Mpi`): opm-grid couples MPI to **Zoltan** (parallel partitioning). Zoltan isn't in vcpkg, so the harness builds the Zoltan package only from a Trilinos clone against MS-MPI, into a separate `build-mpi\` / `install-mpi\` tree. The serial tree is untouched.
- **OpenMP** (`-OpenMP`): uses MSVC's `/openmp:llvm` (OpenMP 3.1+); the default `/openmp` (2.0) rejects OPM's `size_t` parallel-for counters. Needs `libomp140.x86_64.dll` at runtime.
- The two switches are independent and compose into a **hybrid** binary.

2.7 Source patches (outside OPM)

Only DUNE needs source patches (auto-applied after checkout): `gmshreader.hh` (POSIX `ftello/fseeko` → `_ftelli64/_fseeki64`, serial + MPI); and two MPI-only patches dropping the `MPI 3.0` gate and guarding `MPI_CXX_*_COMPLEX` (MS-MPI is MPI-2 level). Trilinos/Zoltan and all vcpkg packages build unmodified. The OPM Windows/MSVC fixes live on the `GitPaean/*` `windows` branch until merged upstream.

3. What was executed

Target: the "fully complete" build the guide defines — `-Mpi -OpenMP -SimTarget all -Upscaling` — i.e. every `flow_*` variant plus the opm-upscaling tools, across all four OPM repos, from `C:\opm\opm_flow_windows`.

1. **Installed toolchain** (winget): VS 2022 Build Tools (VCTools + Win11 SDK 22621 + CMake + Ninja), MS-MPI runtime + SDK.
2. **Cloned** vcpkg, DUNE 2.10.0 (×4), and all 4 OPM repos on the `GitPaean` `windows` branch.
3. **Capped parallelism at 8 jobs** (`build-module.ps1`; OPM's template-heavy TUs are memory-hungry — 8 jobs suit a ≥ 32 GB machine).
4. **Ran the one-shot build:**

```
.\build-all.ps1 -Mpi -OpenMP -SimTarget all -Upscaling -OpmOrg GitPaean -OpmBranch windows
```

This completed end-to-end: vcpkg deps → DUNE (patched) → Zoltan (from Trilinos) → opm-common → opm-grid → opm-simulators (every `flow_*` variant) → opm-upscaling.

Then, to cover **all** targets (tests + examples, which the harness disables by default), each OPM module was reconfigured with `-DBUILD_TESTING=ON -DBUILD_EXAMPLES=ON` and its `all` target rebuilt with ninja keep-

going (`-k 0`), so a single failing target could not abort a module. This is packaged as the new `build-optests.ps1` helper.

4. Findings

4.1 Results — all targets build

OPM repository	Configuration	Executables	Failing
opm-common	testing + examples ON	240	0
opm-grid	testing + examples ON	74	0
opm-simulators	testing + examples ON, target `all`	173	0
opm-upscaling	testing + examples ON, target `all`	31	0
Total		**528**	**0**

The core (non-test) build alone produces 48 `flow_*` / `flowexp_*` simulator variants in opm-simulators and 24 upscaling tools in opm-upscaling; enabling tests + examples raises the totals above. Libraries produced: `opmcommon.lib, opmgrid.lib, zoltan.lib` and the four DUNE libs in `install-mpi\lib`.

4.2 Verification

- `flow_blackoil.exe` runs → `flow 2026.10-pre`, exit 0 (22 MB).
- Native **x64 PE32+**; dependents include `msmpi.dll` + `libomp140.x86_64.dll` — confirming a true hybrid MPI+OpenMP binary — plus MSVCP140/VCRUNTIME140/KERNEL32. **No** cygwin/msys.
- Unit tests run and pass (e.g. `test_2dtables.exe`: 14 cases, no errors; `test_msim_ACTIONX.exe`: 8/8 cases, 321/321 assertions).

4.3 Simulation runs

Norne (`NORNE_ATW2013.DATA`, real-field black-oil case, 247 report steps) — runs fine as a **hybrid MPI + OpenMP** simulation with the default linear solver (CPRW):

```
mpiexec -n 4 flow_blackoil.exe NORNE_ATW2013.DATA --threads-per-process=2 --output-
dir=windows_run
```

```
Using 4 MPI processes with 2 OMP threads on each
 Newton its= 3, linearizations= 4 (0.2sec), linear its= 2 (0.2sec)
 Newton its= 2, linearizations= 3 (0.1sec), linear its= 4 (0.1sec)
```

Healthy nonlinear/linear convergence throughout; the same case also runs serially and with the `ilu0` / `dilu` solver choices.

SPE1 two-phase (`SPE1CASE2_2P.DATA` from opm-tests) with a non-blackoil variant and an explicitly AMG-based solver — full run to completion:

```
flow_oilwater.exe SPE1CASE2_2P.DATA --linear-solver=cpr --output-dir=<dir>
```

```
Overall Newton Iterations:    247    (Wasted: 0; 0.0%)
Overall Linear Iterations:    134    (Wasted: 0; 0.0%)
```
— exit 0.

Together these exercise the default CPR(W) preconditioner, plain AMG/CPR, and the one-level solvers, in serial, MPI, and hybrid MPI+OpenMP modes, across two simulator variants.

4.4 Unit-test suites (ctest, all four modules)

The full test suites were run with `ctest -j 8` in each module's `build-mpi` tree (tests + examples built via `build-optests.ps1`). **Every test that executes passes**; the failures that remain are environmental, not code defects:

Module	Passed	Remaining failures (all environmental)
opm-common	222 / 227 (98%)	4 policy-blocked, 1 needs symlinked test data
opm-grid	79 / 81 (98%)	2 policy-blocked
opm-simulators	62 / 138	68 shell-script tests, 8 policy-blocked
opm-upscaling	2 / 20	17 shell-script tests, 1 policy-blocked

The three environmental categories:

- **Windows "Smart App Control" policy blocks** (ctest shows `BAD_COMMAND`): on machines with SAC in enforce mode, freshly built unsigned executables may be refused; SAC has no exclusion mechanism, and every rebuild produces new file hashes. Disable it on build machines (Windows Security → App & browser control) or accept a fluctuating blocked subset.
- **Symlinked test data** (`ParserIncludeTests`): the repo contains git symlinks, which Windows only materializes with Developer Mode enabled (Settings → For developers) and `core.symlinks=true` at clone time.
- **Shell-script-driven tests** (`run-vtu-test.sh`, `run-compareUpscaling.sh`): cannot run under Windows ctest as-is; an upstream CMake change (bash launcher or UNIX gating) is needed. The underlying simulators these scripts drive are all built and run fine.

Sources must be checked out byte-exact: `build-all.ps1` clones with `-c core.autocrlf=false`, since git-for-Windows' default CRLF conversion corrupts the formatted Eclipse test data. Windows Firewall prompts for each exe's first MPI socket are pre-authorized by running `allow-firewall.ps1` once (elevated).

4.5 Key takeaways

> 1. **Build from a short ASCII path outside sync-managed folders** (e.g. `C:\opm\...`) — vcpkg fails on non-ASCII path components, and sync clients can lock files mid-build.
> 2. **The harness is sound.** The documented one-command build produced every simulator and tool with no source changes beyond the pre-applied DUNE patches and the `windows`-branch OPM fixes.
> 3. **All 528 targets build** — simulators, tools, examples, and unit tests — and **every test that executes passes** (98% pass rate in opm-common/opm-grid; the rest is blocked only by Windows policy, symlink checkout, or POSIX shell-script runners — see §4.4).
> 4. **Memory-bounded:** the whole build ran at 8 parallel compile jobs.

5. GUI front ends: flow-gui and flow-gui-qt

`flow` is a command line program, which is inconvenient as the primary interface on Windows. The harness therefore ships two small GUI front ends (modeled on the basic functionality of OPMRUN from opm-utilities), letting users queue simulations and watch them run without a terminal:

	`flow-gui` (FLTK)	`flow-gui-qt` (Qt 6)
Platforms	Windows, Linux	Windows, Linux, macOS
Toolkit source	FLTK 1.4.3 fetched + built statically at configure time (no package needed)	System Qt on Linux/macOS; official prebuilt Qt (aqtinstall) on Windows
Binary	single static executable	executable + Qt libraries (`windeployqt` on Windows)
Extras	—	persistent settings; the intended base for extension (Qwt/Qt Charts plots, PRT viewer)

Shared functionality: a job queue of `*.DATA` decks; simulator picker (auto-detects `flow` from the harness build tree — any `flow_*` variant works); MPI rank count and OpenMP `--threads-per-process`; output directory policy (per-deck `<deck>_run` or a chosen directory — the chooser can create new folders); extra flow options; a live log of the running simulation; and a Stop button that kills the *entire* MPI process tree (Windows Job Object / `taskkill /T`; POSIX process groups).

```
# Windows (from a setup-env.ps1 shell)
cmake -S flow-gui -B build-gui -G Ninja -DCMAKE_BUILD_TYPE=Release
cmake --build build-gui                       # → build-gui\flow-gui.exe

# Linux: both are two commands after installing X11 headers (flow-gui) or
# qt6-base-dev (flow-gui-qt); see each folder's README for the exact lines.
```

Windows note: building Qt *from source* (e.g. vcpkg's `qtbase`) executes its own freshly compiled `moc` / `rcc` tools, which Smart App Control blocks where enforced — use the official prebuilt (signed) Qt via `aqtinstall` instead, as described in `flow-gui-qt/README.md`. Both GUIs build and launch on Windows, and `flow-gui` has driven real simulation runs; the Linux/macOS builds use standard mechanisms but were not exercised in this session.

6. Reproduce / run

```
# From C:\opm\opm_flow_windows
.\build-all.ps1 -Mpi -OpenMP -SimTarget all -Upscaling -OpmOrg GitPaean -OpmBranch windows

# Optional: also build every unit test + example target, then run the suites
.\build-optests.ps1
ctest -j 8    # from each build-mpi\<module> directory

# One-time (elevated): pre-authorize the exes in Windows Firewall for MPI runs
.\allow-firewall.ps1

# Run a parallel simulation (4 ranks x 2 OpenMP threads each):
mpiexec -n 4 .\build-mpi\opm-simulators\bin\flow_blackoil.exe `
    <deck>.DATA --threads-per-process=2 --output-dir=<dir>
```

Outputs: `build-mpi\opm-simulators\bin\` (simulators + tests), `build-mpi\opm-upscaling\bin\` (tools + tests), `install-mpi\` (libraries + headers). Logs: `build-hybrid.log`, `build-tests-*.log`.